

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

1. Giao tiếp Đồng bộ — REST, gRPC & Resilience

“Microservices favor smart endpoints and dumb pipes. The communication infrastructure should be simple; intelligence belongs in the services.” — Sam Newman, Building Microservices [4a]

1.1. Bạn sẽ học được gì

- Hiểu các kiểu tương tác (interaction styles) trong microservices
- So sánh REST và gRPC — khi nào dùng cái nào
- Sử dụng OpenFeign để gọi service-to-service một cách declarative
- Nắm vững Service Discovery và Load Balancing với Eureka
- Áp dụng các resilience patterns: Circuit Breaker, Retry, Timeout, Bulkhead
- Phân tích bài toán dispatch SQL và tích hợp ngoài trong KBLab

1.2. Giao tiếp đồng bộ — Khi nào nên, khi nào tránh?

1.2.1. Request-Response: mô hình quen thuộc

Giao tiếp đồng bộ (*synchronous communication*) là mô hình quen thuộc nhất với hầu hết developer: service A gửi request đến service B, **chờ** response, rồi xử lý tiếp. HTTP/REST là protocol phổ biến nhất cho mô hình này.

Giao tiếp đồng bộ — Core Service blocked
Client
API Gateway
Core Service
Judge Service
POST /submissions
Forward request
POST /judge/execute [sync call]

BLOCKED
(chờ Judge)

Result: score=85
200 OK + score
Response to client

⚠ Vấn đề: Core bị block toàn bộ thời gian Judge xử lý (có thể vài giây). Thread bị giữ, không phục vụ request khác.

Hình 4.1: Giao tiếp đồng bộ — Core Service blocked trong khi chờ Judge response

1.2.2. Ưu và nhược điểm

Bảng 4.1: Ưu và nhược điểm của giao tiếp đồng bộ

	Ưu điểm	Nhược điểm
Đơn giản	Model request-response quen thuộc, dễ debug	Caller bị block cho đến khi nhận response
Nhất quán	Biết ngay kết quả (thành công/thất bại)	Temporal coupling : cả hai service phải online cùng lúc
Tooling	HTTP tooling phong phú (Postman, curl, browser)	Cascading failures : service B down → A cũng down
Tracing	Request ID để trace qua chuỗi calls	Latency accumulation : $A \rightarrow B \rightarrow C = \text{tổng latency}$

Richardson phân tích rõ trong [2a, Ch.3]: giao tiếp đồng bộ tạo **temporal coupling** (cả hai service phải sẵn sàng cùng lúc) và **runtime dependency** (service gọi không thể hoạt động nếu service được gọi down). Đây là lý do microservices thường ưu tiên async cho các flow không cần response ngay lập tức.

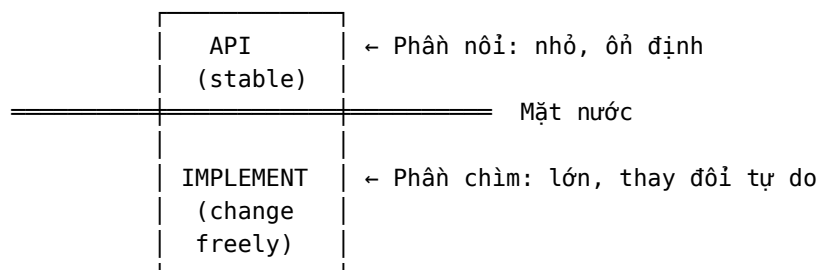
1.2.3. Design-time Coupling vs Runtime Coupling

Richardson trong phiên bản thứ hai [2b, Ch.4] phân biệt rõ hai loại coupling cơ bản — hiểu sai sự khác biệt này là nguyên nhân phổ biến nhất dẫn đến thiết kế microservices kém:

Bảng 4.1b: Hai loại coupling trong microservices

	Design-time Coupling	Runtime Coupling
Định nghĩa	Thay đổi service A → phải thay đổi service B	Service A cần B đang chạy để xử lý request
Khi nào xảy ra	Khi phát triển, compile, deploy	Khi hệ thống đang chạy (runtime)
Ảnh hưởng	Giảm team autonomy, tăng coordination	Giảm availability, tăng latency
Ví dụ KBLab	Core Service đổi DTO format → Auth Service phải cập nhật parser	Core Service gọi sync Judge → Judge down → Core trả lỗi
Giải pháp	API versioning (§3.2), backward compatible changes	Async messaging (Ch.5), Circuit Breaker (§4.4)

Iceberg Principle — Richardson gọi đây là nguyên tắc nền tảng cho loose coupling [2b, Ch.4]:



API (phần nổi) phải nhỏ nhất có thể — chỉ expose những gì consumer thực sự cần. Implementation (phần chìm) có thể lớn và thay đổi tự do mà không ảnh hưởng consumer. **Đây cũng là lý do dùng DTO pattern (§3.4)** thay vì trả entity trực tiếp — entity là “phần chìm” không nên lộ ra.

💡 Tip — DRY và Coupling: Nghịch lý trong Distributed Systems

Trong monolith, DRY (Don't Repeat Yourself) gần như luôn đúng. Nhưng trong microservices, DRY có thể tạo coupling không mong muốn. Ví dụ: `OrderTotalCalculator` dùng chung qua shared library → mỗi lần update business rule → tất cả services dùng library phải upgrade đồng loạt (lock-step deployment). Đôi khi **duplication có chủ đích** tốt hơn coupling — đặc biệt khi logic có thể diverge hợp lệ giữa các contexts. Richardson trong [2b, Ch.4] khuyên: chỉ DRY khi divergent implementations gây **bugs**, không phải mọi trường hợp.

1.2.4. Khi nào dùng sync?

Giao tiếp đồng bộ phù hợp khi:

- **Cần response ngay** — query data (GET), validation, authentication
- **Flow đơn giản** — chuỗi call ngắn ($A \rightarrow B$), không phải chuỗi dài ($A \rightarrow B \rightarrow C \rightarrow D$)
- **Read-heavy operations** — lấy thông tin, không phải thay đổi trạng thái phức tạp

Không phù hợp khi:

- **Long-running operations** — chậm bài SQL mất 5–30 giây
- **Fire-and-forget** — gửi notification, log event
- **Cross-service transactions** — cần saga pattern (Chương 6)

💡 Tip — Quy tắc ngón tay cái

Nếu caller *phải biết* kết quả để xử lý tiếp → sync. Nếu caller chỉ cần *trigger* hành động và không cần kết quả ngay → async (Chương 5). Richardson trong [2a, Ch.3] minh họa rõ: gọi sync để *validate* (cần kết quả ngay), nhưng gửi async để *process* (không cần chờ). Trong KBLab, query Judge health dùng sync, nhưng submit bài chấm dùng async (Kafka) — hai nhu cầu khác nhau.

1.2.5. Interaction Styles — Phân loại kiểu tương tác

Richardson phân loại interaction styles theo hai chiều [2a, Ch.3]:

Bảng 4.2: Interaction styles trong microservices theo Richardson [2a, Ch.3]

	One-to-one	One-to-many
Synchronous	Request/Response	—
Asynchronous	Async request/response, One-way notification	Publish/Subscribe, Publish/ Async responses

Hầu hết service dùng kết hợp nhiều kiểu. Ví dụ: KBLab Core Service dùng *request/response* (Feign) để query Judge, *publish/subscribe* (Kafka) để gửi submissions, và *one-way notification* (WebSocket) để push kết quả.

1.2.6. REST vs gRPC — Hai lựa chọn cho sync

REST (HTTP/JSON) là lựa chọn phổ biến nhất, nhưng không phải duy nhất. **gRPC** — framework RPC mã nguồn mở của Google — là lựa chọn phổ biến thứ hai cho giao tiếp đồng bộ giữa microservices [2a, Ch.3].

Bảng 4.3: REST vs gRPC — so sánh chi tiết

Tiêu chí	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Format	JSON (text, human-readable)	Protocol Buffers (binary, compact)
Performance	Chậm hơn (text parsing)	Nhanh hơn 2-10x (binary + HTTP/2 multiplexing)
Contract	OpenAPI spec (optional)	.proto file (bắt buộc)
Code gen	Optional (OpenAPI generators)	Built-in (protoc generates client/server)
Browser support	Native	Cần gRPC-Web proxy
Streaming	Không native (phải dùng SSE/WebSocket)	Bi-directional streaming native
Debugging	Dễ (curl, Postman, browser)	Khó (cần gRPC tools)

gRPC Architecture — Tại sao nhanh hơn? gRPC nhanh hơn REST không chỉ nhờ binary encoding. Ba yếu tố kiến trúc quan trọng:

1. **HTTP/2 multiplexing** — Nhiều requests cùng dùng một TCP connection (không cần mở connection mới mỗi request). Quan trọng khi service A gọi service B hàng trăm lần/giây.
2. **Protocol Buffers** — Schema-first, binary encoding. Payload nhỏ hơn JSON 3-5x. Schema evolution built-in (backward/forward compatible) — .proto file là contract.
3. **Four communication patterns:** Unary (request-response, như REST), Server streaming (server gửi stream), Client streaming (client gửi stream), **Bi-directional streaming** (cả hai gửi stream đồng thời). Bi-directional streaming rất phù hợp cho real-time use cases — ví dụ: Judge Service stream kết quả từng test case cho Frontend trong khi sinh viên vẫn đang xem.

Khi nào dùng gRPC thay REST?

- **Internal service-to-service:** performance quan trọng, không cần browser → gRPC
- **Public API / Frontend:** cần browser support, human debugging → REST
- **Streaming:** cần real-time bi-directional → gRPC (hoặc WebSocket)
- **Polyglot:** team dùng nhiều ngôn ngữ → gRPC (code gen đa ngôn ngữ)

❗ Phân tích — KBLab không còn là hệ thống REST thuần

LMS core của KBLab vẫn ưu tiên REST/JSON vì browser, dashboard và OpenAPI tooling cần tính dễ debug. Nhưng Network Lab chủ động dùng SOAP/REST/gRPC/JNP để sinh viên thấy khác biệt protocol trong cùng một domain bài tập. Vì vậy, migration sang gRPC không phải mục tiêu “đổi toàn bộ REST”; cách đúng hơn là **chọn protocol theo boundary**: REST cho client-facing API, gRPC cho internal high-throughput/streaming nếu có nhu cầu, và batch/job queue cho các tác vụ judge dài hoặc stateful.

1.2.7. Resilience Metrics — Đo lường độ tin cậy

Trước khi áp dụng resilience patterns (§4.4), cần hiểu cách **đo lường** resilience. Bốn metrics cốt lõi:

Bảng 4.4: Resilience metrics cốt lõi

Metric	Định nghĩa	Ví dụ KBLab
MTBF (Mean Time Between Failures)	Thời gian trung bình giữa hai lần sự cố	Judge Service crash trung bình 1 lần/tuần → MTBF = 168h
MTTR (Mean Time To Recovery)	Thời gian trung bình để khôi phục	Trung bình 15 phút để phát hiện + restart → MTTR = 15min
MTTF (Mean Time To Failure)	Thời gian trung bình từ recovery đến failure tiếp theo	MTTF = MTBF - MTTR
Availability	% thời gian system hoạt động	$= \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$

Availability “nines”:

Bảng 4.5: Availability “nines” — downtime cho phép theo target

Target	Downtime/năm	Downtime/tháng	Phù hợp cho
99% (two nines)	3.65 ngày	7.3 giờ	Internal tools, dev environments
99.9% (three nines)	8.76 giờ	43.8 phút	KBLab production (phù hợp)
99.99% (four nines)	52.6 phút	4.38 phút	E-commerce, banking
99.999% (five nines)	5.26 phút	26.3 giây	Critical infrastructure

❗ Nguyên tắc — Error Budgets

Google SRE (Site Reliability Engineering) đề xuất: nếu target availability = 99.9%, hệ thống có **error budget** = **0.1%** downtime/tháng (43.8 phút). Dùng error budget cho: deployments, experimentation, controlled maintenance. Khi hết budget → freeze deployments, focus stability. Đây là cách biến “availability target” từ khái niệm mơ hồ thành **nguyên tắc vận hành cụ thể** (xem thêm SLI/SLO/SLA ở Ch.11).

1.3. OpenFeign — Declarative REST Client

1.3.1. Vấn đề: gọi service khác không đơn giản như gọi hàm

Trong monolith, gọi module khác là một dòng code: `judgeService.execute(request)`. Trong microservices, cùng logic đó trở thành: xây dựng HTTP request, serialize payload, gắn headers (JWT token), gửi qua network, xử lý response codes, deserialize result, handle timeout/error.

1.3.2. OpenFeign: gọi REST như gọi hàm

OpenFeign (Spring Cloud OpenFeign) cho phép khai báo API call như một Java interface — framework tự động generate implementation:

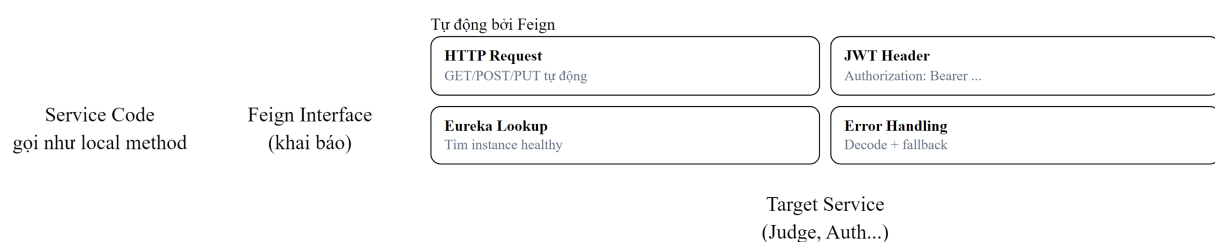
Listing 4.1: So sánh `RestTemplate` (verbose) và `OpenFeign` (declarative)

```
// ❌ Manual REST call – verbose, boilerplate
RestTemplate restTemplate = new RestTemplate();
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(jwtToken);
HttpEntity<JudgeRequest> entity = new HttpEntity<>(request, headers);
ResponseEntity<JudgeResult> response = restTemplate.exchange(url, HttpMethod.POST,
entity, JudgeResult.class);

// ✅ Declarative Feign client – clean, type-safe
@FeignClient(name = "judge", url = "${feign.judge.url}")
public interface JudgeClient {
    @PostMapping("/api/judge/sql")
    JudgeResult submitSql(@RequestBody JudgeRequest request); // Gọi như hàm bình
thường!
}
```

1.3.3. Cách hoạt động

Feign Proxy — Tự động hóa HTTP request



```
@FeignClient(name = "judge-service")
public interface JudgeClient {
    @PostMapping("/api/judge/execute")
    JudgeResult execute(@RequestBody SubmitRequest req);
}
```

Hình 4.2: Cách Feign proxy tự động xử lý HTTP request, JWT, load balancing

Feign tự động xử lý: serialization/deserialization (JSON ↔ Java), header injection (JWT token qua `RequestInterceptor`), service discovery (Eureka lookup thay vì URL hardcoded), và error decoding (response code → exception).

1.4. Service Discovery & Load Balancing

1.4.1. Vấn đề: URL cứng trong distributed system

Trong monolith, gọi module khác là gọi hàm — không cần biết “module ở đâu”. Trong microservices, mỗi service là một process riêng, chạy trên host:port khác nhau. Câu hỏi: **service A tìm service B ở đâu?**

Cách đơn giản nhất: hardcode URL trong config. Nhưng khi service scale (3 instances của Judge Service), URL nào? Khi service di chuyển (deploy lên container mới), IP thay đổi — ai cập nhật?

1.4.2. Service Discovery patterns

Có hai pattern chính [2a, Ch.3]:

Client-side discovery — Client (service gọi) tự query service registry để tìm danh sách instances, rồi tự load balance:

Client-side Discovery — Service tự query Eureka
Core Service
Eureka Registry
Judge #1
:8081
Judge #2
:8082
1. Query: judge-service instances?
2. [judge:8081, judge:8082]
3. Client-side LB
Round-Robin chọn instance
4. POST /judge/execute
5. Result (score=85)

Client-side LB: Core Service chọn instance (Round-Robin). Spring Cloud LoadBalancer / Ribbon thực hiện điều này. Ưu điểm: Không có hop trung gian. Nhược điểm: Client cần tích hợp thư viện LB.

Hình 4.3: Client-side discovery — service tự query Eureka và load balance

Server-side discovery — Load balancer (hoặc API Gateway) đứng giữa, client chỉ cần biết URL của load balancer:

Server-side Discovery — Load Balancer điều hướng
Core Service
Load Balancer
AWS ALB / Nginx / k8s
Judge #1
Judge #2
1. POST /judge/execute
2. Chọn instance
query Service Registry
3. Forward to healthy instance
4. Result
5. Forward response

LB là trung gian: AWS ALB, Nginx, Kubernetes Service. Client không cần biết instance nào. Trade-off: Thêm 1 hop mạng, nhưng client đơn giản hơn — không cần tích hợp thư viện LB.

Hình 4.4: Server-side discovery — load balancer điều hướng request

1.4.3. Netflix Eureka trong KBLab

KBLab sử dụng **Netflix Eureka** cho cụm Java/Spring services — client-side discovery pattern:

Bảng 4.6: Thành phần Eureka trong KBLab

Thành phần	Vai trò	Port
eureka-registry	Service registry server	9000
Mỗi service	Eureka client (tự đăng ký)	—
Spring Cloud LoadBalancer	Client-side load balancing	—

Khi microservice khởi động, nó đăng ký tại Eureka server với tên (`spring.application.name`) và địa chỉ. Gateway và các service khác query Eureka để tìm instances. Trong config Gateway: `uri: lb://core-service` nghĩa là tra cứu Eureka tìm tất cả instances có tên `core-service`, rồi load balance giữa chúng.

❗ Phân tích — Service naming không nhất quán

Trong snapshot cũ của KBLab, `core-service` và `assignment-service` từng dùng `spring.application.name = "app"` — vi phạm nguyên tắc unique service identity [2a]. Khi Eureka nhận hai services cùng tên, nó coi chúng là instances của *cùng một service* → routing sai. Đây là hậu quả của việc Assignment tách ra từ Core nhưng không đổi service name. **Migration path:** đổi thành `kblab-core` và `kblab-assignment`, cập nhật Eureka config và Gateway routes.

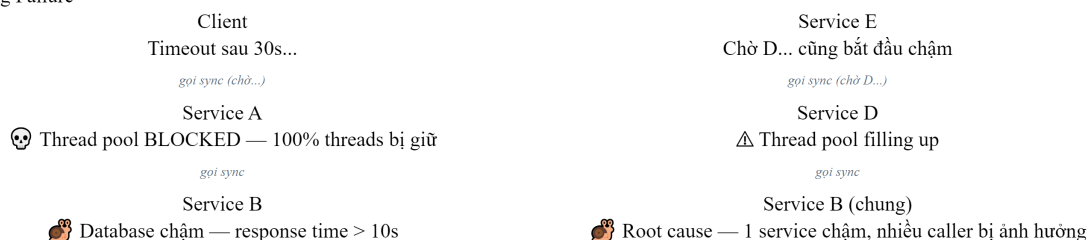
1.5. Resilience Patterns — Sống sót trong Distributed System

1.5.1. Tại sao cần resilience?

Trong monolith, nếu database chậm, *toàn bộ* ứng dụng chậm — nhưng ít nhất lỗi rõ ràng. Trong microservices, khi service B chậm, service A *vẫn chạy* nhưng threads bị block chờ B → requests đến A cũng bị chậm → service C gọi A cũng bị chậm → **cascading failure** lan khắp hệ thống [5, Ch.7].

Hugo Rocha trong [5] nhấn mạnh: trong distributed system, *lỗi không phải ngoại lệ* — *lỗi là trạng thái bình thường*. Network sẽ timeout, services sẽ crash, databases sẽ chậm. Câu hỏi không phải “nếu lỗi xảy ra” mà là “khi lỗi xảy ra”.

Cascading Failure



B chậm → A bị block → Client timeout
B chậm → D bị ảnh hưởng → E cũng bị ảnh hưởng

Giải pháp: **Circuit Breaker** (fig 4.6) — fail fast thay vì chờ vô hạn · **Bulkhead** (fig 4.7) — cách ly failure, không lan truyền · **Timeout** — giới hạn thời gian chờ

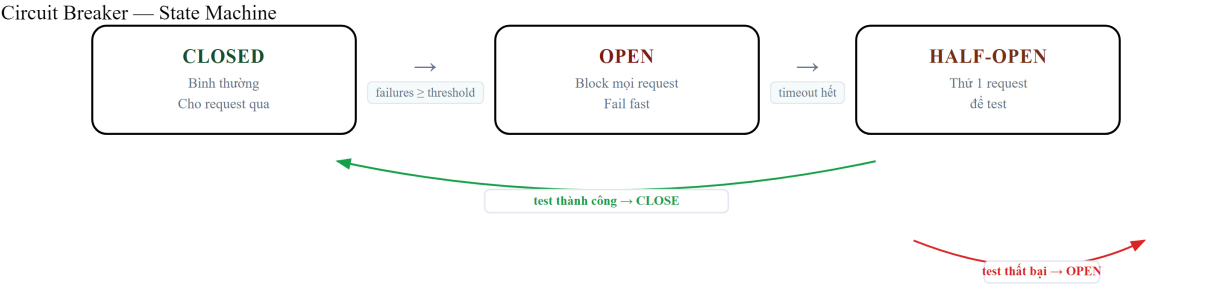
Hình 4.5: Cascading failure — Service B chậm làm A, D, E đều bị ảnh hưởng

1.5.2. Bốn resilience patterns cốt lõi

Bảng 4.7: Bốn resilience patterns cốt lõi

Pattern	Nguyên lý	Tương tự	Ví dụ KBLab
Timeout	Đặt giới hạn thời gian chờ. Không bao giờ chờ vô hạn	Hẹn giờ nấu ăn — không chờ vô hạn	Feign call timeout 3s cho Judge
Retry	Tự động thử lại khi gặp lỗi tạm thời	Gọi điện lại khi tín hiệu kém	Retry 3 lần với exponential backoff
Circuit Breaker	Ngắt mạch khi service downstream liên tục lỗi	Cầu dao điện — ngắt khi quá tải	Judge down → trả fallback message
Bulkhead	Cách ly resources theo service/function	Khoang tàu thủy — 1 khoang ngập, tàu không chìm	Thread pool riêng cho Judge vs Question

1.5.2.1. Circuit Breaker — Chi tiết state machine



Cấu hình Circuit Breaker (Resilience4j)

```
failureRateThreshold = 50% | slidingWindowSize = 10 | waitDurationInOpenState = 30s
```

LMS: @CircuitBreaker annotation trên JudgeClient.execute()

Hình 4.6: Circuit Breaker state machine — Closed, Open, Half-Open

Khi circuit ở trạng thái **Open**, mọi request được reject *ngay lập tức* — không gửi tới service downstream. Điều này bảo vệ cả caller (không block threads) và callee (không bị overwhelm khi đang recovery).

1.5.2.2. Bulkhead — Cách ly resources

Bulkhead Pattern — Thread Pool cách ly

Core Service

→ Judge requests

→ Question requests

→ Contest requests



Judge pool đầy → chỉ Judge bị ảnh hưởng. Question và Contest vẫn hoạt động bình thường = Cách ly failure, không lan truyền.

❌ Không có Bulkhead

1 pool chung 45 threads. Judge chậm → tất cả block.

✅ Có Bulkhead

Pool riêng. Judge chậm → chỉ Judge bị ảnh hưởng, Question/Contest vẫn OK.

```
@Bulkhead(name = "judge", type = THREADPOOL, maxConcurrentCalls = 10)
```

Hình 4.7: Bulkhead pattern — thread pool cách ly theo function

Tên “Bulkhead” lấy từ thiết kế tàu thủy: khoang tàu bị nước tràn vào, các khoang khác vẫn khô — tàu không chìm. Michael Nygard giới thiệu pattern này trong *Release It!* (2007) [5, Ch.7].

1.5.2.3. Lưu ý quan trọng cho Retry

Chỉ retry idempotent operations (GET, PUT, DELETE). Retry POST có thể tạo duplicate data. Richardson trong [2a, Ch.3] nhấn mạnh: retry token (idempotency key) là bắt buộc khi retry non-idempotent operations.

1.5.2.4. Kết hợp các patterns

Trong thực tế, các patterns thường kết hợp theo thứ tự:

Request → Timeout (3s) → Retry (3 lần, backoff) → Circuit Breaker → Bulkhead → Actual Call

Spring Cloud Circuit Breaker + Resilience4j là stack phổ biến nhất cho Java microservices. Chỉ cần thêm dependency và annotation — không cần viết logic retry/circuit breaker thủ công.

1.5.2.5. Resilience4j Implementation — Code ví dụ cho LMS

Listing 4.2: Resilience4j annotations cho Judge Feign Client

```
// Core Service – gọi Judge với đầy đủ resilience patterns
@Service
public class JudgeCallService {
    private final MySQLClient mysqlClient;

    @CircuitBreaker(name = "judge", fallbackMethod = "judgeFallback")
    @Retry(name = "judge")
    @TimeLimiter(name = "judge")
    @Bulkhead(name = "judge")
    public CompletableFuture<JudgeResult> executeSQL(JudgeRequest request) {
        return CompletableFuture.supplyAsync(() ->
            mysqlClient.execute(request)
        );
    }

    // Fallback khi circuit OPEN hoặc tất cả retries thất bại
    private CompletableFuture<JudgeResult> judgeFallback(
        JudgeRequest request, Throwable t) {
```

```
        log.warn("Judge unavailable: {}, submissionId={}",
            t.getMessage(), request.getSubmissionId());
        return CompletableFuture.completedFuture(
            JudgeResult.pending("Sandbox đang bận, bài sẽ được chấm khi sẵn sàng")
        );
    }
}
```

Listing 4.3: Resilience4j YAML configuration cho LMS

```
# application.yml – Resilience4j configuration
resilience4j:
  circuitbreaker:
    instances:
      judge:
        slidingWindowSize: 10           # Đánh giá trên 10 requests gần nhất
        failureRateThreshold: 50        # Mở circuit khi 50% requests lỗi
        waitDurationInOpenState: 30s    # Chờ 30s trước khi thử lại (half-open)
        permittedNumberOfCallsInHalfOpenState: 3 # Thử 3 calls khi half-open
        recordExceptions:               # Chỉ count những exceptions này
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - feign.FeignException.ServiceUnavailable

  retry:
    instances:
      judge:
        maxAttempts: 3                 # Tối đa 3 lần thử
        waitDuration: 1s               # Chờ 1s giữa các lần
        exponentialBackoffMultiplier: 2 # 1s → 2s → 4s (exponential)
        retryExceptions:
          - java.io.IOException        # Chỉ retry lỗi tạm thời
        ignoreExceptions:
          - com.lms.exception.BusinessException # KHÔNG retry lỗi logic

  timelimiter:
    instances:
      judge:
        timeoutDuration: 5s           # Timeout 5s cho mỗi call

  bulkhead:
    instances:
      judge:
        maxConcurrentCalls: 10        # Tối đa 10 calls đồng thời đến Judge
        maxWaitDuration: 500ms        # Chờ tối đa 500ms nếu bulkhead đầy
```

Bảng 4.8: Giá trị recommend cho từng environment

Parameter	Development	Production (LMS)	High-traffic
slidingWindowSize	5	10	20
failureRateThreshold	50%	50%	30%
waitDurationInOpenState	10s	30s	60s
retry.maxAttempts	1	3	3
timelimiter.timeoutDuration	10s	5s	3s
bulkhead.maxConcurrentCalls	5	10	25

💡 Tip — Thứ tự áp dụng annotations

Resilience4j xử lý annotations theo thứ tự **ngoài vào trong**: Bulkhead → TimeLimiter → CircuitBreaker → Retry → Actual Call. Nghĩa là: request phải qua bulkhead trước (kiểm tra concurrent limit), rồi timeout, rồi circuit breaker check, rồi retry nếu lỗi. Thứ tự này quan trọng — nếu Retry bao ngoài CircuitBreaker, retry sẽ thử lại ngay cả khi circuit đã OPEN (vô nghĩa). Cấu hình mặc định của Resilience4j đã xử lý đúng thứ tự [5, Ch.7].

📌 Nguyên tắc — Design for Failure

“In distributed systems, failure is not an exception — it’s a normal state. Design every inter-service call assuming it will fail. Timeout, Retry, Circuit Breaker, Bulkhead — these are not optional, they are prerequisites.”

— *Tổng hợp từ Hugo Rocha [5, Ch.7] và Michael Nygard, Release It!*

1.6. Case Study: KBLab SQL Judge — Coordinator và DBMS Workers

1.6.1. Bài toán

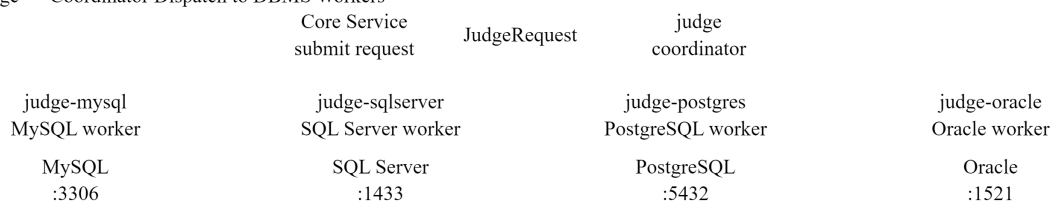
KBLab hỗ trợ sinh viên thực hành SQL trên nhiều DBMS. Mỗi DBMS chạy trong sandbox container riêng biệt. Khi sinh viên nộp bài, hệ thống cần:

1. Xác định DBMS nào (dựa vào câu hỏi)
2. Gửi SQL đến đúng sandbox service
3. So sánh kết quả với đáp án (SHA-256 hash)
4. Trả về kết quả

1.6.2. Hiện trạng: Strategy Pattern + OpenFeign

KBLab implement bài toán này bằng một service điều phối **judge** kết hợp các worker **judge-*** theo từng DBMS:

SQL Judge — Coordinator Dispatch to DBMS Workers



Ownership: Core gọi judge; judge sở hữu routing DBMS và worker contract.

Scale: Thêm DBMS mới = thêm một worker judge-* + registry config.

Resilience: Circuit Breaker riêng theo worker. MySQL down không ảnh hưởng SQL Server.

Migration: Phase 2: Async queue cho job dài — không block Core khi worker chậm.

Hình 4.8: SQL Judge coordinator — dispatch SQL đến các worker **judge-*** theo DBMS

Core Service không nên biết chi tiết MySQL hay SQL Server được xử lý ở worker nào. Nó gửi yêu cầu chấm đến **judge**; **judge** đóng vai trò coordinator: nhận database type, chọn worker tương ứng (**judge-mysql**, **judge-sqlserver**, ...), gọi worker qua HTTP/Feign hoặc queue, rồi chuẩn hóa kết quả trả về Core. Đây là pattern phổ biến: một coordinator giữ contract nghiệp vụ ổn định, các processor phía sau chuyên môn hóa theo runtime/DBMS.

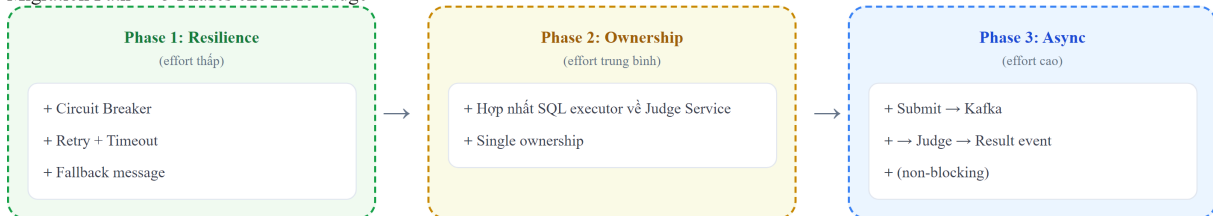
1.6.3. Phân tích các vấn đề

Bảng 4.9: Phân tích vấn đề SQL Judge coordinator/worker

#	Vấn đề	Mô tả	Best Practice
1	Routing ownership chưa rõ	Nếu Core cũng chứa logic chọn DBMS, routing bị rò rỉ ra ngoài Judge boundary	Single ownership: judge là coordinator duy nhất
2	Magic IDs / hardcoded routing	Database type xác định bằng hardcoded UUID/string	Enum hoặc configuration-driven registry
3	Resilience theo worker chưa rõ	judge-mysql down không nên kéo hỏng toàn bộ judge hoặc worker khác	Circuit Breaker/Timeout riêng cho từng worker
4	Fallback chưa nhất quán	Worker DBMS lỗi → user nhận lỗi kỹ thuật hoặc submission stuck	Fallback message, retry/queue, status rõ ràng
5	Worker contract chưa chuẩn hóa	Mỗi judge-* dễ trả format/result khác nhau	Chuẩn hóa JudgeRequest/JudgeResult contract

1.6.4. Đề xuất migration

Migration Path — 3 Phases cho LMS Judge



Khuyến nghị: Bắt đầu từ Phase 1 — ROI cao nhất, không cần thay đổi kiến trúc.

Phase 3: Chỉ thực hiện khi submission volume tăng và blocking trở thành bottleneck thực sự.

Hình 4.9: Lộ trình migration cho SQL Judge — từ resilience đến async

- **Phase 1 — Làm rõ ownership** (ưu tiên cao, effort thấp): Core chỉ gọi judge; mọi routing DBMS nằm trong judge coordinator. Không để routing DBMS rò sang Core hoặc các service không sở hữu nghiệp vụ chấm SQL.
- **Phase 2 — Thêm resilience theo worker** (ưu tiên cao): @CircuitBreaker, @Retry, @TimeLimiter riêng cho từng judge-*. Khi judge-mysql down → trả fallback: “Sandbox MySQL đang bận, bài sẽ được chấm khi sẵn sàng”, không ảnh hưởng judge-sqlserver.
- **Phase 3 — Chuẩn hóa worker contract** (effort trung bình): Tất cả worker nhận cùng JudgeRequest, trả cùng JudgeResult, routing config-driven thay vì hardcoded IDs.
- **Phase 4 — Chuyển flow dài sang async** (effort cao, giá trị lớn): Submit → Kafka/queue → judge coordinator → judge-* worker → Result event → Core cập nhật. User không

chờ đồng bộ — nhận notification khi có kết quả. Đây chính là pattern mà KBLab đã bắt đầu implement (Chương 5).

1.6.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice

Không phải mọi tích hợp trong KBLab đều nên đi qua cùng một protocol. Với QLĐT hoặc GitHub Classroom, KBLab nên đặt **Anti-Corruption Layer** ở boundary: adapter dịch contract bên ngoài thành command/event nội bộ, che giấu format, lỗi và rate limit của hệ thống đối tác. Với Network Practice, điểm đáng học là ngược lại: context này được thiết kế độc lập để đánh giá giao tiếp mạng, không phụ thuộc vào service chấm SQL. Vì vậy, nó là ví dụ tốt cho bounded context theo protocol, còn vấn đề coordinator/worker cần phân tích nằm ở SQL Judge.

📌 Nguyên tắc — Sync → Async là hành trình phổ biến

Nhiều hệ thống bắt đầu với sync (đơn giản, dễ debug) rồi chuyển sang async khi cần scale. Richardson trong [2a] minh họa hành trình này: bắt đầu với REST calls giữa services, sau đó chuyển sang Saga khi cần distributed transaction. KBLab đang ở giữa hành trình: một số flow đã async (Kafka cho submission), một số vẫn sync (Feign cho query), và SQL Judge cần tách rõ coordinator/worker trước khi scale ngang an toàn. Chương 5 sẽ tiếp tục phần async.

⚠️ Lưu ý —

1. **Không đặt timeout cho inter-service calls** — Mặc định nhiều HTTP client chờ vô hạn (hoặc 30-60 giây). Hậu quả: khi service downstream chậm, threads caller bị block → cascading failure lan khắp hệ thống. *Phòng tránh*: luôn cấu hình timeout rõ ràng (Netflix chuẩn: 1-3 giây cho internal calls).
2. **Tin tưởng rằng network luôn reliable** — Viết code như thể mọi HTTP call đều thành công. Hậu quả: lỗi đầu tiên ở production mới phát hiện không có retry, không fallback. *Phòng tránh*: áp dụng resilience patterns từ đầu (Circuit Breaker + Retry + Timeout) — ngay cả khi chưa cần scale (§4.4).
3. **Retry mà không kiểm tra idempotency** — Retry POST requests tạo duplicate data (hai submissions cho cùng một lần nộp). Hậu quả: dữ liệu sai, user bị tính điểm trùng. *Phòng tránh*: chỉ retry idempotent operations (GET, PUT, DELETE), hoặc dùng idempotency key cho POST.

1.7. Tổng kết

Giao tiếp đồng bộ là mô hình đơn giản nhất nhưng tiềm ẩn rủi ro lớn nhất trong distributed system. Temporal coupling, cascading failures, và latency accumulation là ba vấn đề cốt lõi mà developer phải đối mặt.

OpenFeign đơn giản hóa service-to-service calls thành Java interface declarations, loại bỏ boilerplate HTTP code. Service Discovery (Eureka) giải quyết bài toán “tìm service ở đâu” — một vấn đề không tồn tại trong monolith nhưng quan trọng sống còn trong microservices.

Resilience patterns — Timeout, Retry, Circuit Breaker, Bulkhead — không phải optional. Chúng là điều kiện tiên quyết để hệ thống microservices hoạt động ổn định trong production.

Phân tích KBLab cho thấy thiếu resilience patterns là gap nghiêm trọng nhất trong giao tiếp đồng bộ.

Ở Chương 5, chúng ta sẽ khám phá **giao tiếp bất đồng bộ** — giải pháp cho những giới hạn của sync: Apache Kafka, event-driven architecture, và cách KBLab sử dụng messaging cho submission pipeline.

1.8. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, Service Discovery
2. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
3. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.7: Resilience & Reliability

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Coupling Taxonomy (design-time vs runtime), Iceberg Principle, DRY in distributed systems; Ch.9: IPC patterns
5. Michael Nygard, *Release It!*, 2nd Ed. (2018) — Circuit Breaker, Bulkhead, Timeout patterns

Nguồn trực tuyến:

- [W3] Netflix Technology Blog — “Making the Netflix API More Resilient” (2011)
- Resilience4j documentation — resilience4j.readme.io
- Spring Cloud OpenFeign reference — docs.spring.io

Tài liệu tham khảo